

Static Analysis

Software Testing & Quality Assurance

Dr. Karim Akilal

February 12, 2026

Université de Béjaïa

- ✓ Qu'est ce que l'analyse statique?
- ✓ Différence entre analyse statique et analyse dynamique.
- ✓ Quelques bugs que l'analyse statique peut détecter.
- ✓ Outils d'analyse statique pour le C.
- ✓ Intégrer l'analyse statique dans votre *workflow*.

Qu'est-ce que l'analyse statique?

Définition

L'analyse statique *examine* le code, **sans l'exécuter**, à la recherche de *bugs* potentiels, de vulnérabilités (sécurité), et de divers problèmes de qualité.

Pourquoi est-ce important?

- Gestion manuelle de la mémoire (en C par exemple).
- Pas de vérifications à l'exécution (limites, null pointers)
- Undefined behavior (UB).
- ...

Analyse statique vs. Analyse dynamique

STATIQUE

- Code at rest (Non exécuté)
- Pas d'exécution nécessaire.
- Rapide et *scalable*.
- **Exemples:**
 - GCC warnings
 - Cppcheck
 - Clang analyzer

DYNAMIQUE

- Code en cours d'exécution.
- Plus lent, runtime cost (mémoire, CPU, etc)
- **Exemples:**
 - Unit tests
 - Valgrind
 - AddressSanitizer

Exemple #1: Guess the bug!

```
int* create_array(int size) {  
    int arr[size];  
    for (int i = 0; i < size; i++) {  
        arr[i] = i * 2;  
    }  
    return arr;  
}
```

Réponse #1: Return pointer to local var.

```
int* create_array(int size) {  
    int arr[size];  
    for (int i = 0; i < size; i++) {  
        arr[i] = i * 2;  
    }  
    return arr;    // ! BUG!  
}
```

// GCC -Wall et Clang Static Analyzer signalent ça immédiatement!

Code Compiler Warnings

GCC -Wall -Wextra

Clang -Weverything

Lint Tools

Cppcheck, Splint

Flexelint

Deep Analyzers

Clang Static Analyzer

Coverity, PVS-Studio

ShieldAlt Security Scanners

Flawfinder, RATS

SonarQube

Exemple #2: Guess the bug!

```
void my_copy(char* input) {  
    char buffer[10];  
    strcpy(buffer, input);  
    printf("%s\n", buffer);  
}
```


Réponse #2: Buffer Overflow

Flawfinder et RATS signalent les unsafe functions:

```
void my_copy(char* input) {  
    char buffer[10];  
    strcpy(buffer, input);    // ! No bounds check  
    printf("%s\n", buffer);  
}  
  
// Better: Use strncpy or strcpy_s
```

Exemple #3: Guess the bug!

```
char* greeting = malloc(100);  
if (greeting == NULL) return -1;  
  
strcpy(greeting, "Hello, ");  
strcat(greeting, name);  
printf("%s\n", greeting);
```

Réponse #3: Memory Leak

```
char* greeting = malloc(100);  
if (greeting == NULL) return -1;  
  
strcpy(greeting, "Hello, ");  
strcat(greeting, name);  
printf("%s\n", greeting);  
  
// ! Missing: free(greeting);
```

Exemple #4: Guess the bug!

```
void process_user(User* user) {  
    printf("ID: %d\n", user->id);  
}
```

```
User* u = malloc(sizeof(User));  
if (u == NULL) return -1;  
process_user(u);  
process_user(NULL);
```

Réponse #4: Déréférencement d'un pointeur NULL

```
void process_user(User* user) {  
    printf("ID: %d\n", user->id); // ! No null check!  
}  
  
User* u = malloc(sizeof(User));  
if (u == NULL) return -1;  
process_user(u); // OK here  
process_user(NULL); // ! Crash!
```

Exemple #5: Guess the bug!

```
int sum_items(int items) {  
    int sum;  
  
    if (items > 0) {  
        sum = items * 10;  
    }  
  
    return sum;  
}
```

Réponse #5: Variable non initialisée

```
int sum_items(int items) {  
    int sum; // ! Not initialized!  
  
    if (items > 0) {  
        sum = items * 10;  
    }  
  
    return sum; // ! Undefined if items <= 0  
}
```

Exemple #6: Guess the bug!

```
int* alloc_array(int count, int size) {  
    int total = count * size;  
    return malloc(total);  
}
```


Réponse #6: Integer Overflow

```
int* alloc_array(int count, int size) {  
    int total = count * size; // ! Overflow!  
    return malloc(total);  
}
```

```
// If count = 1000000, size = 10000:  
// Result overflows, allocates tiny buffer  
// Later writes cause buffer overflow
```

Exemple #7: Guess the bug!

```
char* data = malloc(100);  
strcpy(data, "Hello");  
printf("%s\n", data);  
  
free(data);  
  
printf("%s\n", data);  
data[0] = 'h';
```

Réponse #7: Use After Free

```
char* data = malloc(100);  
strcpy(data, "Hello");  
printf("%s\n", data);  // OK  
  
free(data);  
  
printf("%s\n", data);  // ! Use after free!  
data[0] = 'h';  // ! Undefined behavior
```

Exemple #8: Guess the bug!

```
void cleanup(Resource* r1, Resource* r2) {  
    if (r1 != NULL) {  
        free(r1);  
    }  
    if (r2 != NULL) {  
        free(r2);  
    }  
}
```

Réponse #8: Double Free

```
void cleanup(Resource* r1, Resource* r2) {  
    if (r1 != NULL) {  
        free(r1);  
    }  
    if (r2 != NULL) {  
        free(r2); // ! If r1 == r2!  
    }  
}
```

// Corrupts heap, crash or exploit

Exemple #9: Guess the bug!

```
void initialize(int* array, int size) {  
    for (int i = 0; i <= size; i++) {  
        array[i] = 0;  
    }  
}
```

```
int data[10];  
initialize(data, 10);
```

Réponse #9: Off-by-One Error

```
void initialize(int* array, int size) {  
    for (int i = 0; i <= size; i++) { // ! Should be <  
        array[i] = 0;  
    }  
}  
  
int data[10];  
initialize(data, 10); // ! Writes to data[10]!
```

Exemple #10: Guess the bug!

```
void log_message(char* user_input) {  
    printf(user_input);  
}
```


Réponse #10: Format String Vulnerability

Security issue with printf-family functions:

```
void log_message(char* user_input) {  
    printf(user_input); // ! DANGEROUS!  
}  
  
// Attacker sends: "%x %x %x %s"  
// Leaks memory contents!  
  
// CORRECT:  
printf("%s", user_input);
```

Exemple #11: Guess the bug!

```
int process(int value) {  
    if (value < 0) {  
        return -1;  
    } else {  
        return value * 2;  
    }  
  
    printf("Processing\n");  
    return 0;  
}
```

Réponse #11: Dead Code

```
int process(int value) {  
    if (value < 0) {  
        return -1;  
    } else {  
        return value * 2;  
    }  
  
    printf("Processing\n"); // ! Never reached!  
    return 0;  
}
```

Exemple #12: Guess the bug!

```
int read_data(unsigned int len) {  
    char buffer[100];  
  
    if (len > sizeof(buffer)) {  
        return -1;  
    }  
}
```

Réponse #12: Sign Conversion Issue

```
int read_data(unsigned int len) {  
    char buffer[100];  
  
    if (len > sizeof(buffer)) { // ! Problem!  
        return -1;  
    }  
}  
  
// -1 becomes 4294967295 unsigned!
```

En gros, que détecte l'analyse statique?

Problèmes de mémoire

- Memory leaks
- Buffer overflows
- Use after free
- Double free

Erreurs logiques

- Null pointer dereferences
- Uninitialized variables

Problèmes de Sécurité

- Format string vulnerabilities
- Integer overflow
- Race conditions

Qualité de code

- Dead code
- Sign conversion issues
- Off-by-one errors

Outils populaires d'analyse statique en C

Outil	Type	Meilleur Pour
Avertissements GCC/Clang	Compilateur	Erreurs de base, première ligne de défense
Cppcheck	Open source	Bugs généraux, gratuit et rapide
Clang Static Analyzer	Analyse approfondie	Analyse sensible aux chemins d'exécution
Coverity	Commercial	Entreprise, systèmes critiques
Splint	Basé sur des annotations	Vérification de la gestion de la mémoire
Flawfinder/RATS	Sécurité	Détection de fonctions dangereuses
Frama-C	Vérification formelle	Preuve de la correction

Intégrer ces outils dans votre *workflow*

Development Stages:

1. Intégration à l'IDE

Retour en temps réel lors du codage

2. Hooks de pré-commit

Les hooks Git valident avant le commit

3. Pipeline CI/CD

Vérifications automatisées sur les pull requests

4. Analyses planifiées

Analyse approfondie pendant les heures creuses

Dans un Makefile...

```
CC = gcc
CFLAGS = -Wall \
        -Wextra -Werror

analyze:
    cppcheck *.c
    scan-build \
        gcc $(CFLAGS) ...
```


- Activer tous les avertissements du compilateur dès le début (`-Wall` `-Wextra` `-Werror`)
- Utiliser des fonctions sûres (`strncpy`, `snprintf`, variantes avec vérification des limites)
- Toujours vérifier les valeurs de retour (`malloc`, `fopen`, etc.)
- Initialiser toutes les variables lors de leur déclaration
- Utiliser plusieurs outils — chacun détecte des problèmes différents
- Traiter rapidement les avertissements — ne pas les laisser s'accumuler
- Combiner l'analyse statique avec des tests dynamiques

QUESTIONS?